

Software:

- Vivado 2018.2 ,
- SDK 2018.2,
- Serial Port Communication (HERCULES used in this example)

Hardware: ZedBoard

DataFiles:

- **BaseProject**
 - Vivado project that contains all the necessary IP configuration and context switch/restore functionality.
- **RTL_IP**
 - Used to create/test context switching for any RTL based code.
- **ctr_example**
 - An out of the box example that features a 32-bit saturating counter that starts counting up once it is programmed. This example project is generated using **RTL_IP** project folder as well.
 - Contains ready to use bitstream and partial bitstream.

A - Testing the context switching on your custom RTL code/IP.

1. Open the **RTL_IP** project in Vivado 2018.2.
2. Write your RTL code. The figure below shows the example of counter_ip used in this example.

```

22 ;
23 module ctr_example(
24     input          clk,
25     input          rst,
26     input          enable,
27     output reg [31:0] ctr_val
28 );
29
30 always@(posedge clk)
31 begin
32     if (!rst)
33     begin
34         ctr_val <= 'd0;
35     end
36     else
37     begin
38         if (enable && ctr_val < 32'hFFFF_FFFF)
39             ctr_val <=ctr_val+1;
40         else
41             ctr_val<=ctr_val;
42     end
43 end
44 endmodule

```

3. Instantiate your rtl code into top wrapper file called **CUSTOM_IPS** as shown below.

```

`timescale 1 ns / 1 ps

module CUSTOM_IPS
(
    input          clock      ,
    input          reset      ,
    output [7:0] Output_1      ,
    output [31:0]Output_Bus
);
    wire [31:0]ctr_val;

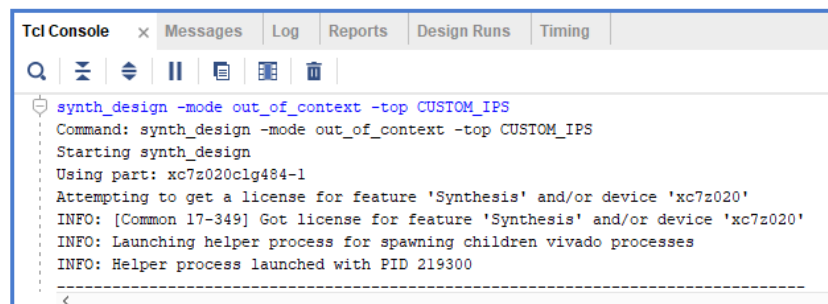
    ctr_example ctr_example(
        .clk      (clock),
        .rst      (reset),
        .enable   (1'b1),
        .ctr_val  (ctr_val)
    );

    assign Output_Bus = ctr_val;
    assign Output_1   = ctr_val[31:24];
endmodule

```

4. Run the following commands in Tcl Console of Vivado.

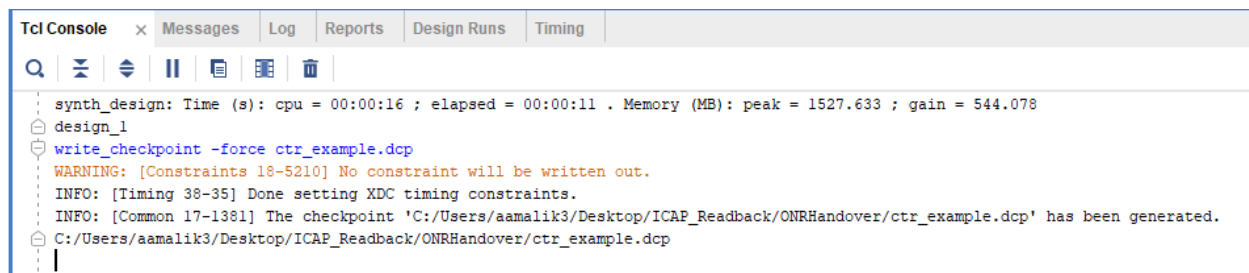
➤ `synth_design -mode out_of_context -top CUSTOM_IPS`



```
Tcl Console x Messages Log Reports Design Runs Timing
synth_design -mode out_of_context -top CUSTOM_IPS
Command: synth_design -mode out_of_context -top CUSTOM_IPS
Starting synth_design
Using part: xc7z020clg484-1
Attempting to get a license for feature 'Synthesis' and/or device 'xc7z020'
INFO: [Common 17-349] Got license for feature 'Synthesis' and/or device 'xc7z020'
INFO: Launching helper process for spawning children vivado processes
INFO: Helper process launched with PID 219300
```

5. Once the design finishes synthesizing, create the design checkpoint that will be used to create partial bitstream file and used in **BaseProject** folder.

➤ `write_checkpoint -force ctr_example.dcp`

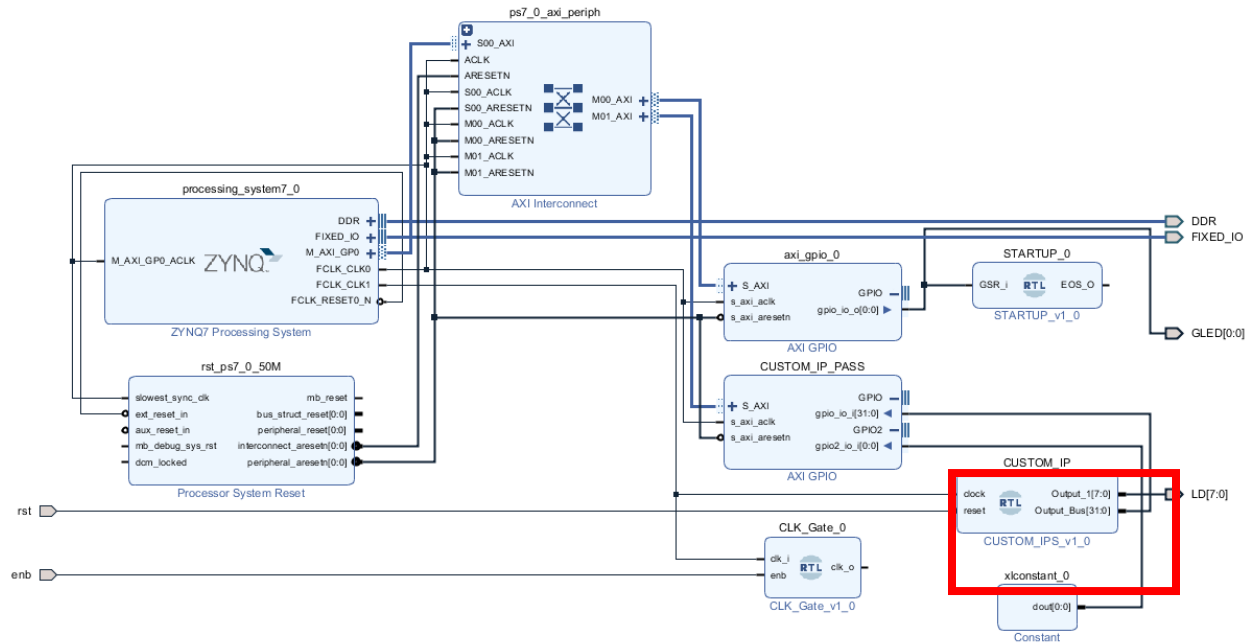


```
Tcl Console x Messages Log Reports Design Runs Timing
synth_design: Time (s): cpu = 00:00:16 ; elapsed = 00:00:11 . Memory (MB): peak = 1527.633 ; gain = 544.078
design_1
write_checkpoint -force ctr_example.dcp
WARNING: [Constraints 18-5210] No constraint will be written out.
INFO: [Timing 38-35] Done setting XDC timing constraints.
INFO: [Common 17-1381] The checkpoint 'C:/Users/aamalik3/Desktop/ICAP_Readback/ONRHandover/ctr_example.dcp' has been generated.
C:/Users/aamalik3/Desktop/ICAP_Readback/ONRHandover/ctr_example.dcp
```

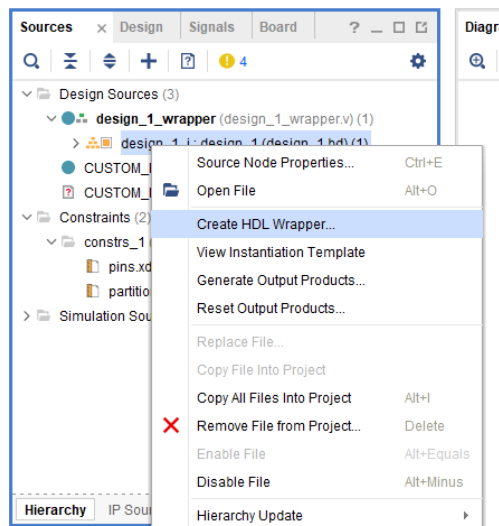
Make sure to note the location where the checkpoint is being created. In the above location it is created at the following location: **C:/Users/aamalik3/Desktop/ICAP_Readback/ONRHandover/**

B - BaseProject to test context switch mechanism

1. The baseproject consist of several IPs that can drive an RTL code to support context switch. The figure shows the block diagram of these IPs connected together.



2. To add/update your own RTL design, user needs to replace the CUSTOM_IP and regenerate the project bitstream. In doing so, make sure the input and output ports have the same name and configuration as set in **Step A-3** above.
3. Once the design has been updated with user specified CUSTOM_IP, validate the design by pressing **F6** key.
4. Once the design has been validated, right click the block design to update the wrapper file as shown below.



5. Enter the following commands in Tcl Console.

- `reset_run synth_1`
- `launch_runs synth_1 -jobs 30`
- `wait_on_run -timeout 10 synth_1`
- `open_run synth_1 -name synth_1`

6. Wait until the design synthesis is complete.

7. Once the design has been synthesized enter the following command in Tcl console.

- `update_design -cell [get_cell design_1_i/CUSTOM_IP/inst] -black_box`

8. At this time, user may get an Error saying that the design is already a black box module. This Error can be safely ignored. Proceed to the next step.

9. Give the project a custom name by entering the following commands in Tcl Console.

- `set benchmarkname "ctr_example"`

10. Import your custom RTL_IP into the BaseProject by running the following tcl command. This is the checkpoint that was created previously in **Step A-5**. Make sure that the correct path is specified where the checkpoint is located.

- `read_checkpoint -cell [get_cell design_1_i/CUSTOM_IP/inst] ./$benchmarkname.dcp`

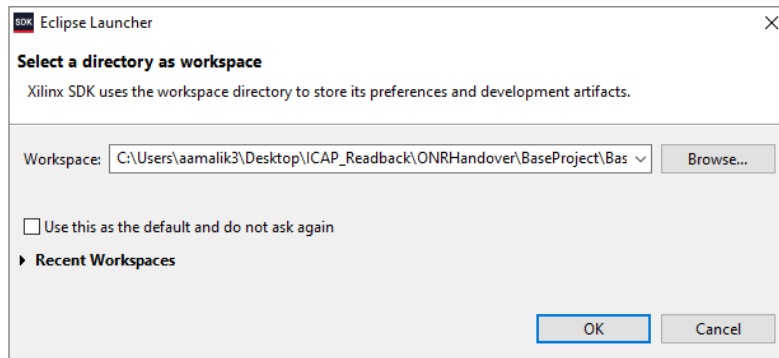
11. Enter the following commands one after another.

- `opt_design`
- `place_design`
- `route_design`
- `file mkdir $benchmarkname`
- `write_bitstream -force -file ./$benchmarkname/$benchmarkname.bit`
- `write_cfgmem -force -format BIN -interface SMAPx32 -disablebitswap -loadbit "up 0 ./$benchmarkname/${benchmarkname}_pblock_custom_ip_partial.bit" ./$benchmarkname/${benchmarkname}_partial.bin`

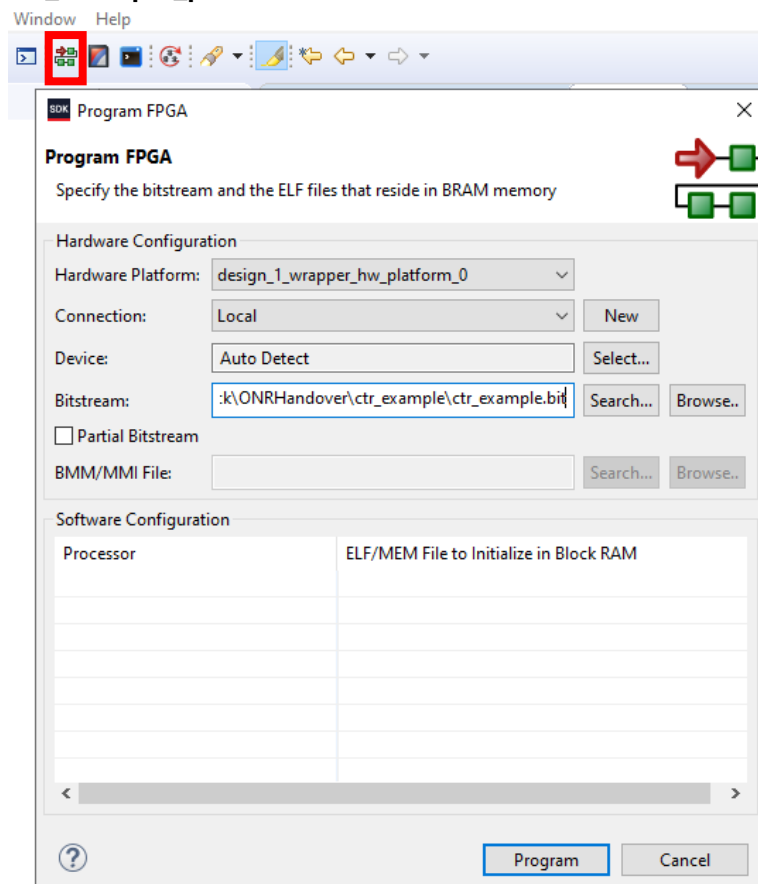
12. If everything was completed successfully a folder name should be created with the same name what was specified in **Step B-9**. In this example it will be `ctr_example`.

C- Running the out-of-the-box ctr_example

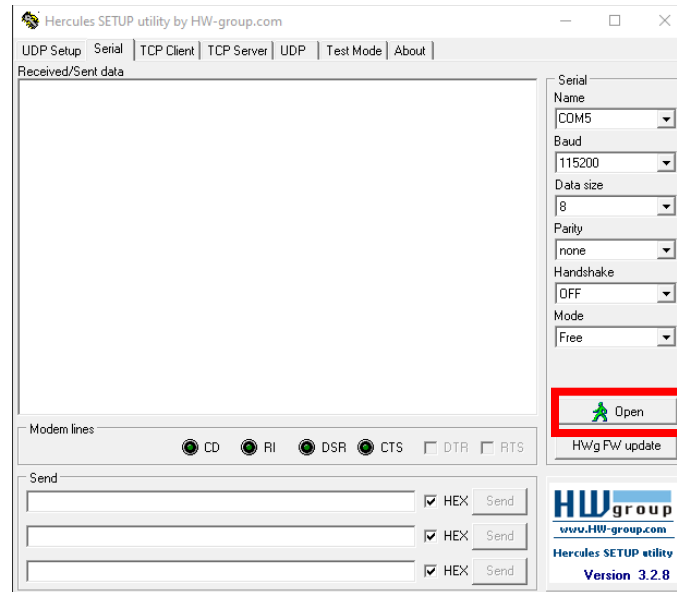
1. Launch the **SDK 2018.2**.
2. Navigate to the following workspace **BaseProject/BaseProject.sdk** and click **OK**.



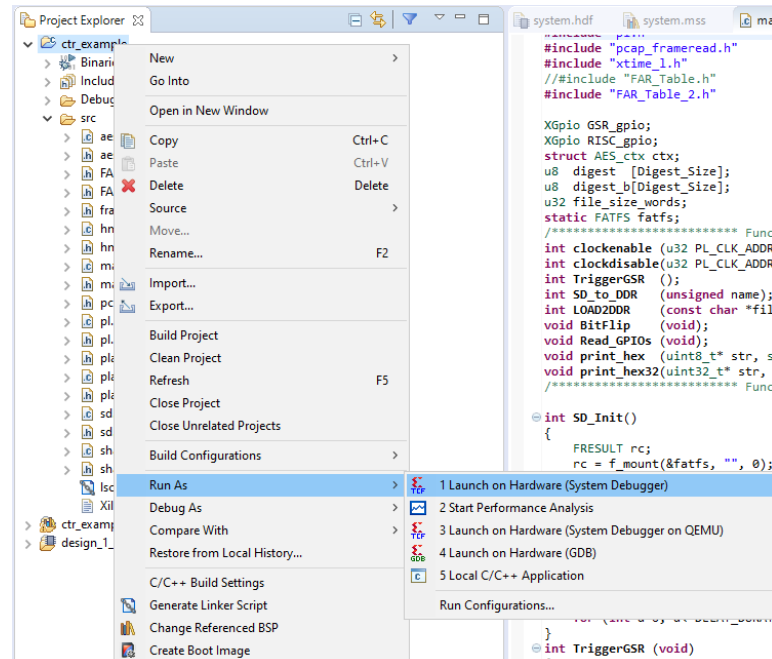
3. Wait until the project workspace finishes initialization.
4. Program the FPGA with the **ctr_example.bit** file and then re-program once using **ctr_example_partial.bit** file.



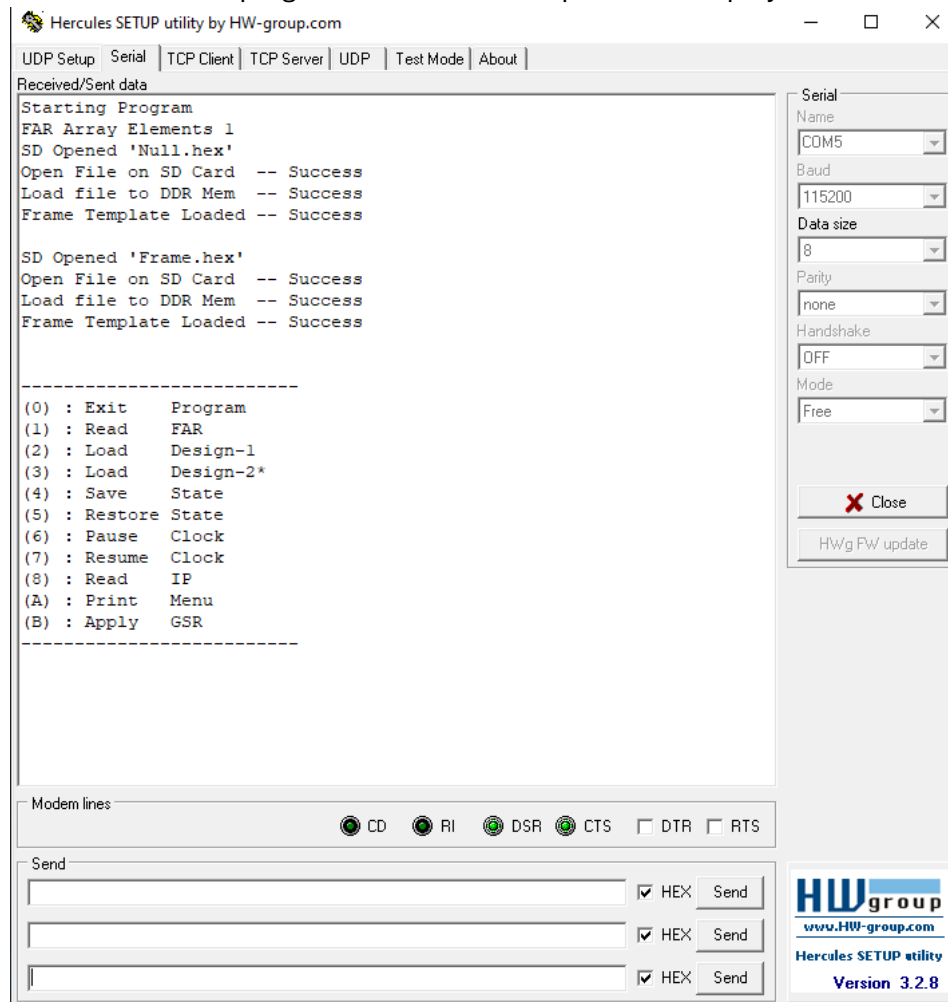
- Place the **ctr_example_partial.bin** partial bit file onto the SD-card and insert into the ZedBoard.
- Open any serial port communication software with following settings. In this example **Hercules** is used for serial communication. Click **Open**.



- Run the **PS C Source** code on the processing system (PS) as shown below.



8. Once the PS is programmed the serial port will display the following menu.



9. User can see the current value of 32-bit counter bybpressing '8' key in Hercules windows. In the figure below the counter shows the initial value of all zeroes.

```

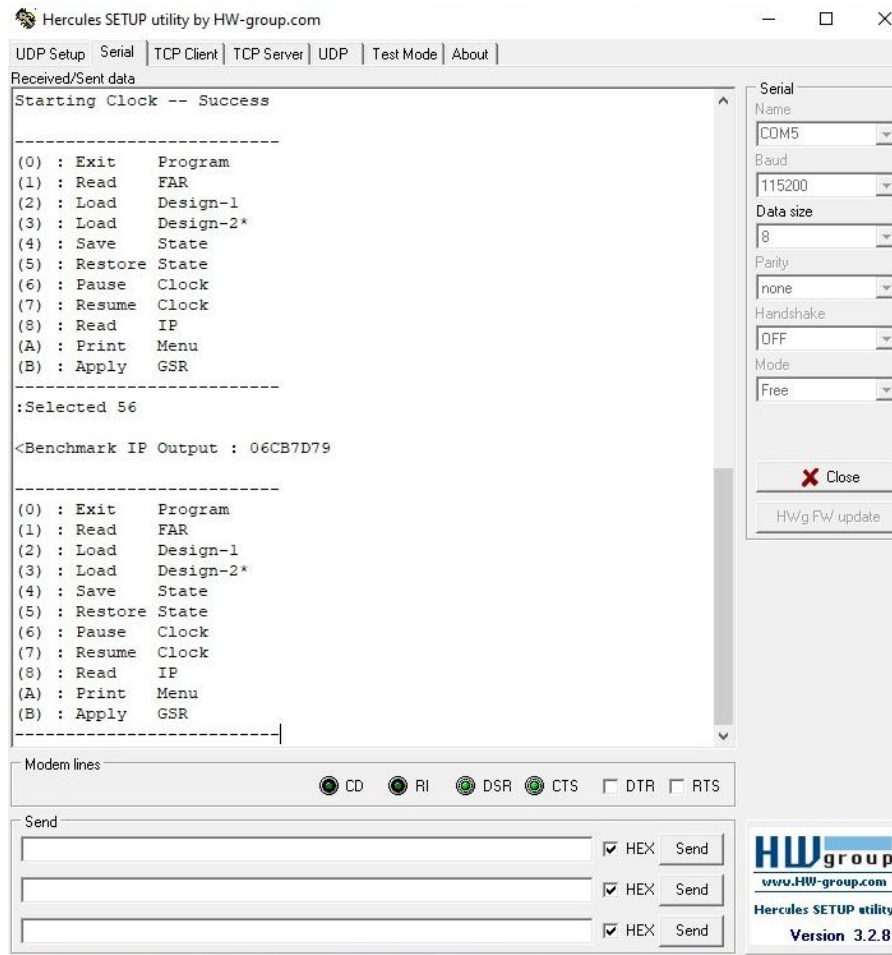
-----
:Selected 56

<Benchmark IP Output : 00000000

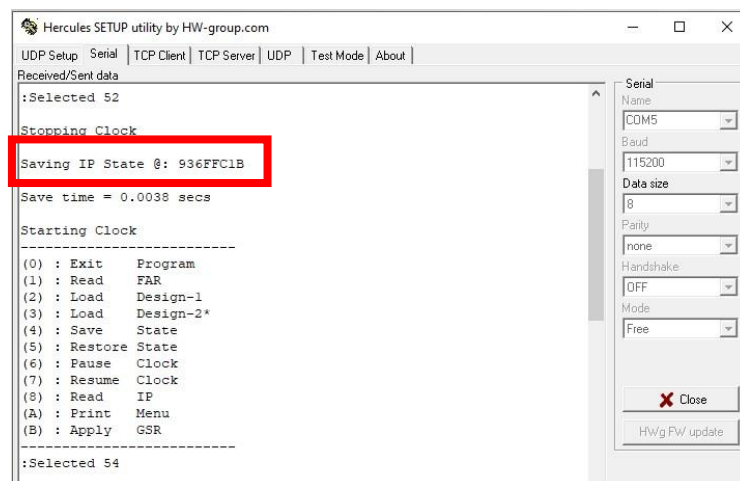
-----
(0) : Exit    Program
(1) : Read    FAR
(2) : Load    Design-1
(3) : Load    Design-2*
(4) : Save    State
(5) : Restore State
(6) : Pause   Clock
(7) : Resume  Clock
(8) : Read    IP
(A) : Print   Menu
(B) : Apply   GSR
-----

```

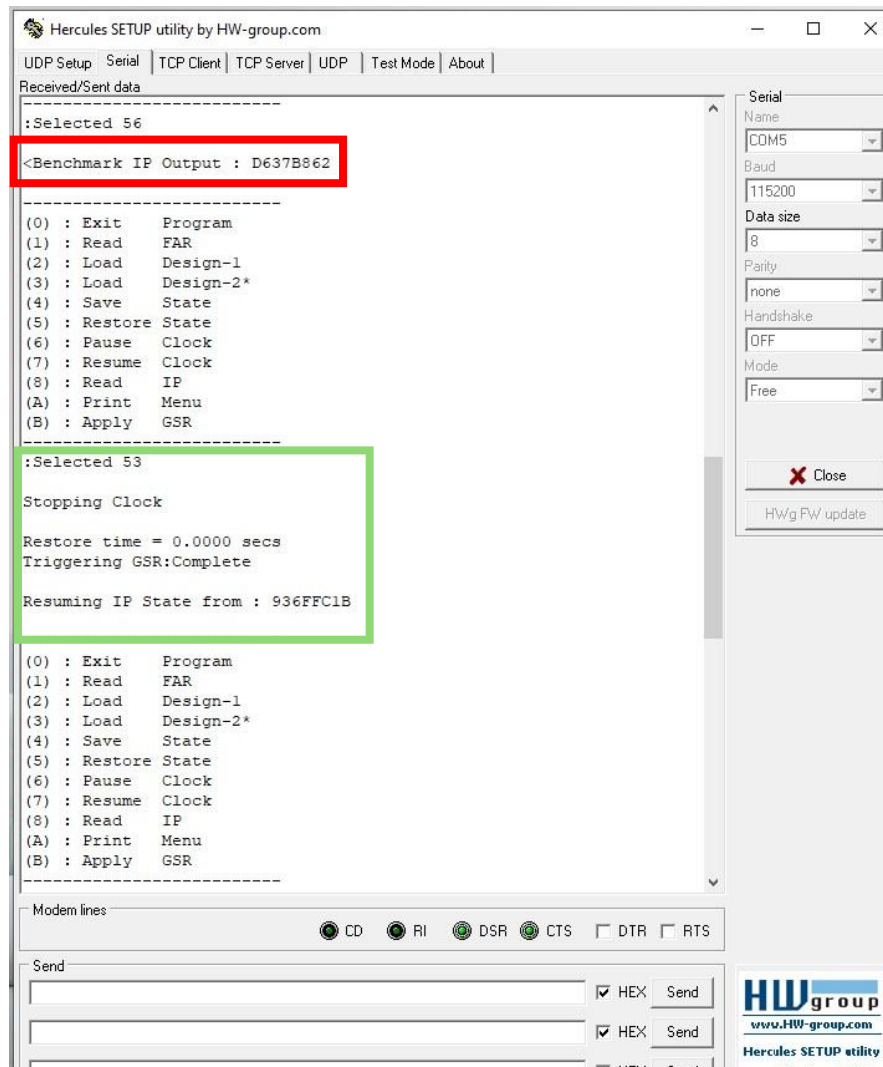

10. After some time the counter will count up to some value as shown below.



11. User can select any arbitrary point in time to save the IP context by pressing the '4' key in Hercules window. This will store the context in the on-chip DRAM while displaying the current value of counter when the context was saved.

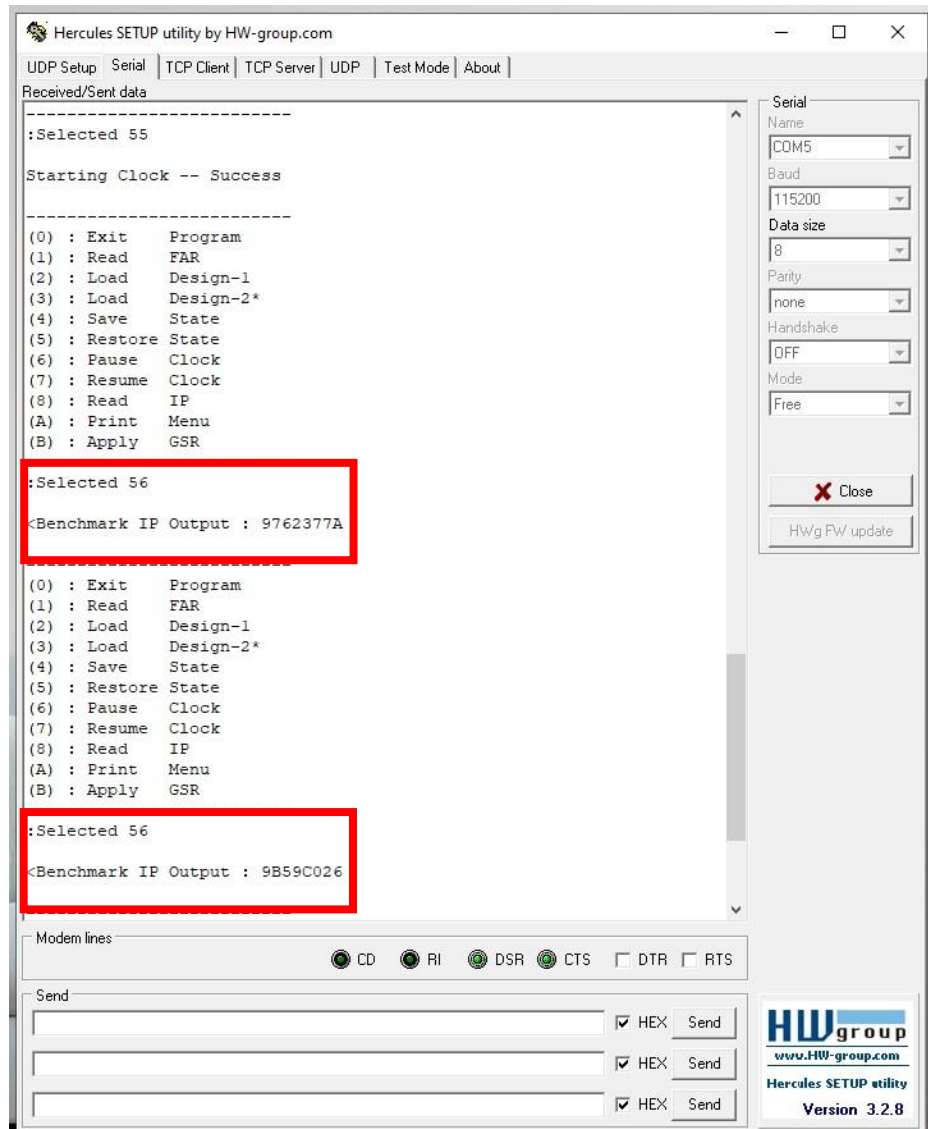


12. Once the context is stored the design keep on running as normal (shown in red box). At any time user can press the '5' key to restore the counter value and resume it from its saved state as shown below in green box.



13. Once the design is restored it will start working as normal. In this example since it is a saturating 32-bit counter it will start counting until it reached the maximum value of **0xFFFFFFFF**. The user can verify the counter is working correctly after restoration by repeatedly pressing the "8" key.

User Guide for Testing Preemption



User Guide for Testing Preemption

